

Analysis: A Whole New Word

Test set 1

Since $L \leq 2$, this test set can be solved using a complete search. First, we collect the letters that appear among the first characters of the input words in a set C_1 and the letters that appear among the second characters of the input words in a set C_2 . Any candidate new word has the form c_1c_2 , where c_1 is in C_1 and c_2 is in C_2 . For each candidate new word, we check whether this word is in the input. We can output any candidate new word which does not appear in the input as our answer. If every candidate new word already appears in the input, the case is impossible.

Since there are only at most 26^2 different candidate words that we need to try, this solution will run very quickly.

Test set 2

In early rounds of Code Jam, a complete search will often work for the first test set, but will generally not work for subsequent test sets. This problem is an exception! Our approach above will work just fine for test set 2.

We can create sets $C_1, C_2, \dots, C_L$ as in the solution above, and then use them to generate candidate words as before, one at a time. If we encounter a word that is not in the input, we can return it as our answer. If it turns out that there are exactly N candidate words (which implies that every word that could be generated is already in the input), the case is impossible. Otherwise, we can be sure that we will have found an answer by the time we generate and check the $(N + 1)$ th candidate word, since there are only N words in the input list.

Analysis: Ant Stack

Test set 1

To solve this test set, we can use [dynamic programming](#) (DP). We define a function $f(x, y)$ as the maximum number of ants that can form a stack (following the stack requirements given in the problem statement), in which we only consider ants from the first ant to the x -th ant, inclusive, and the sum of the ants' weight is not more than y .

We can compute the value of $f(x, y)$ by considering two cases:

- Suppose we don't put the x -th ant on the bottom of the stack. Then we can ignore the x -th ant, and so the value of $f(x, y)$ from this case becomes $f(x - 1, y)$.
- Suppose we put the x -th ant on the bottom of the stack. Then we need to maximize the number of ants to be put on top of the x -th ant, which is the value of $f(x - 1, \min(6W_x, y - W_x))$. Counting the x -th ant as well, the value of $f(x, y)$ from this case becomes $f(x - 1, \min(6W_x, y - W_x)) + 1$. Note that we only consider this case if $y \geq W_x$.

Between the two cases (or only one case if $y < W_x$), we take the larger value as the value of $f(x, y)$.

The answer for the problem is the value of $f(N, \infty)$. Since there are $O(N)$ possible values for x , $O(\max(W))$ possible values for y , and $O(1)$ iterations for each computation of $f(x, y)$, this solution runs in $O(N \times \max(W))$ time.

There is also another solution involving another DP formulation f' , where $f'(x, y)$ only considers ants from the x -th ant to the N -th ant (instead of the first ant to the x -th ant).

Test set 2

To solve this test set, we need to find the value of K , the maximum possible answer to the problem. To have a stack with as many ants as possible, where the upper-bound of the ants' weight is fixed by a constant (i.e. 10^9 in this problem), we greedily put the lightest ant possible on the bottom of the stack. In other words, a stack with as many ants as possible will have ants with weights something like (1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 5, 6, ...). We stop as soon as we need to add an ant with a weight more than 10^9 . By creating a simple script, we can determine that the value of K is 139, much smaller than N .

Therefore, we can solve this test set with another DP formula. We define a function $g(x, y)$ as the minimum sum of the weights of the ants that can form a stack of y ants where only the ants from the first ant to the x -th ant are considered, or ∞ if no such stack exists.

Again, we can compute the value of $g(x, y)$ by considering two cases:

- Suppose we don't put the x -th ant on the bottom of the stack. Then we can ignore the x -th ant, and so the value of $g(x, y)$ from this case becomes $g(x - 1, y)$.
- Suppose we want to put the x -th ant on the bottom of the stack. We first need to check whether this is possible. We can compute the value of $g(x - 1, y - 1)$, the minimum sum of the weights of the ants that can form a stack of $y - 1$ ants where only the ants from the first ant to the $(x - 1)$ -th ant are considered. If $g(x - 1, y - 1) \leq 6W_x$, then it is possible to put the

x-th ant on the bottom of the stack. The value of $g(x, y)$ from this case becomes $g(x - 1, y - 1) + \mathbf{W_x}$.

Between the two cases (or only one case if $g(x - 1, y - 1) > 6\mathbf{W_x}$), we take the smaller value as the value of $g(x, y)$.

The answer for the problem is the largest possible value S where $g(\mathbf{N}, S) < \infty$. Since there are $O(\mathbf{N})$ possible values for x , $O(K)$ possible values for y , and $O(1)$ iterations for each computation of $g(x, y)$, this solution runs in $O(\mathbf{NK})$ time.

Analysis: Lollipop Shop

This problem's acceptance condition is based on [competitive analysis](#). The problem is a bit tough to test locally, even for an interactive problem. It is easy to generate a particular set of customer preferences, but determining the maximum possible number of lollipops we can sell requires [bipartite matching](#). However, we do not necessarily need to do that; since there is only one test set, which is Visible, we have some room for experimentation and for small leaps of faith.

A solution that just picks random legal flavors is not good enough to pass, so we need a couple of insights:

- Selling a lollipop, if possible, is never worse than not selling a lollipop. Assume that there is some good solution that involves not selling a lollipop now, and selling L lollipops in total later. Selling a lollipop now can only stop at most 1 lollipop from being sold later, so we can still sell $L-1$ more lollipops.
- If we have a choice of lollipops to sell, the best we can do is sell a lollipop that has the lowest probability of being liked. Intuitively, this saves the flavors that are more likely to show up later. (This intuition can be proved mathematically.)
- But we don't know the true probability for each flavor. The best estimate we can get for a flavor at any point is the number of customers that have liked that flavor so far, divided by the total number of customers seen so far.

Therefore, our best strategy is to always sell a lollipop if possible; whenever there are multiple flavors to choose from, we choose one of the flavors that we have seen the minimal number of times among previous customers' preferences. In problems like this, when we have to break a tie, we should do so at random just in case the problem setters have tried to anticipate and thwart particular strategies like always choosing the smallest ID number. (In this problem, it would have been impossible for the setters to penalize that, though!)

Notice that our solution would not work for an arbitrary list of probabilities, even with 200 customers. If every probability is 0.02, for example, our strategy loses its power (since all flavors are equally likely), and each flavor appears frequently enough to make our strategy do much worse than matching, but infrequently enough that we have very few chances to make the right choices. However, the problem guarantees that the probabilities are drawn randomly from the range $[0.005, 0.1]$. This should give us enough confidence that the probabilities will be different enough for our strategy to exploit.

The algorithm we have described can be shown to be at least as good as any other solution. But what if we had been less certain? In a problem with only Visible test sets, it is generally better to try (at the risk of a 4 minute penalty) than to spend more than 4 minutes worrying.